

Functional Paradigms in Computer Vision: Pure-Numpy Augmentation and Test-Time Atoms

The Architectural Shift to Pure Functions in Deep Learning

The architectural rigor of machine learning pipelines in competitive environments, particularly across Kaggle Competition Design Guidelines (CDGs), necessitates data augmentation and preprocessing routines that are highly reproducible, framework-agnostic, and computationally efficient. While deep learning frameworks such as TorchVision (operating under a BSD license) and computer vision libraries like Albumentations (operating under an MIT license) offer extensive capabilities, they often introduce stateful dependencies, implicit data type conversions, and hardware-specific non-determinism.

Building a core library of "atoms"—best-in-class, pure-function implementations leveraging foundational numpy and scipy operations—enables the construction of robust, mathematically verifiable directed acyclic graphs (DAGs) for image processing. These atoms are designed for a hypothetical scion-atoms-dl library, focusing on explicit signatures: taking an input ndarray, an explicit set of parameters, and returning an output ndarray without modifying the input in place. This paradigm shift guarantees that operations like fold ensembling, test-time augmentation, and heavy geometric compositions remain strictly deterministic and free from hidden state mutations.

This analysis exhaustively details the mathematical mechanisms, vectorized implementations, and functional contracts for core computer vision augmentation primitives. These primitives serve as the elemental building blocks for high-performance classification, object detection, and semantic segmentation systems across dozens of CDG stages.

Spatial and Information-Dropping Regularizers

Regularization techniques that manipulate spatial structures or combine distinct samples have proven critical in mitigating neural network memorization and sensitivity to adversarial perturbations. By forcing the network to rely on partial information or localized features, these methods drastically improve generalization on unseen test folds.

CutMix: Patch-Based Label Interpolation

CutMix, introduced by Yun et al. (2019), addresses the information loss inherent in standard regional dropout by replacing a deleted spatial patch with a corresponding patch from another image.² Unlike methods that blend images globally, CutMix operates locally, forcing the

network to localize features rather than relying on global context. The canonical implementation is licensed under MIT.⁴

The mathematical formulation relies on a binary mask M indicating the replaced region. The mixed image is calculated as $x_{mix} = M \odot x_a + (1 - M) \odot x_b$, where $\odot$ is element-wise multiplication. The spatial dimensions of the bounding box (r_x, r_y, r_w, r_h) are determined by sampling a combination ratio $\lambda \sim \text{Beta}(\alpha, \alpha)$. The width and height of the crop are proportional to $\sqrt{1 - \lambda}$, ensuring the area ratio of the inserted patch mathematically matches the intended $1 - \lambda$ distribution.

The coordinates are uniformly sampled across the image dimensions (W, H) :

$$r_x \sim U(0, W)$$

$$r_y \sim U(0, H)$$

$$r_w = W\sqrt{1 - \lambda}$$

$$r_h = H\sqrt{1 - \lambda}$$

To maintain functional purity, the bounding box calculation must enforce strict boundary clipping to prevent out-of-bounds indexing. The labels are interpolated proportionally to the exact geometric area of the crop, resolving edge cases where clipping alters the theoretical λ .

Name: cutmix_apply

Description: Apply CutMix augmentation by replacing a random rectangular region of image_a with the same region from image_b, interpolating labels by exact area ratio.

Source: <https://github.com/clovaai/CutMix-PyTorch> (MIT)

Concept type: sampler

Signature: (image_a: NDArray, image_b: NDArray, label_a: NDArray, label_b: NDArray, bbox: tuple[int,int,int,int]) -> tuple

Contracts:

- require: image_a.shape == image_b.shape
- require: 0 <= bbox <= image_a.shape
- require: 0 <= bbox <= image_a.shape
- ensure: result.shape == image_a.shape

Witness: 32x32x3 uint8 images, known bbox, verify pixel replacement and label mix
Dependencies: numpy only
CDG stages covered: bengali_grapheme_1st/heavy_augmentation,
cassava_leaf_1st/image_augmentation, global_wheat_1st/heavy_augmentation

Cutout: Uninformative Region Masking

Preceding GridMask and CutMix, Cutout (DeVries & Taylor, 2017) introduced the concept of zeroing out a random rectangle of the input image. If a model can classify an image correctly even with part of it missing, it has learned redundant, distributed features rather than relying on

a single discriminative patch.⁵ The original paper utilized 16×16 patches on CIFAR-10, though in modern pipelines, cutout sizes of 20-30% of the image dimension are common.

While Cutout is computationally cheap, its primary vulnerability lies in the stochasticity of the crop placement. If the randomly selected region entirely engulfs the single discriminative feature of the image (e.g., the beak of a bird in a fine-grained classification task), the label effectively becomes noise.

Name: cutout_apply

Description: Zero out a random rectangular region of an image to enforce distributed feature learning.

Source: DeVries & Taylor 2017

Concept type: masking

Signature: (image: NDArray, bbox: tuple[int,int,int,int], fill_value: float) -> NDArray

Contracts:

- require: $0 \leq \text{bbox} < \text{bbox} \leq \text{image.shape}$
- require: $0 \leq \text{bbox} < \text{bbox} \leq \text{image.shape}$
- ensure: $\text{result.shape} == \text{image.shape}$
Witness: Generate cutout, verify pixels inside bbox match fill_value and external pixels are unmodified.
Dependencies: numpy only
CDG stages covered: bengali_grapheme_1st/heavy_augmentation

GridMask: Deterministic Structured Deletion

Standard continuous dropout techniques like Cutout frequently drop excessively large continuous regions, leading to the complete removal of the target object, or overly small regions that fail to regularize effectively.⁶ GridMask (Chen, 2020) resolves this by generating a structured grid of dropped squares, ensuring a mathematically guaranteed balance between information retention and deletion.⁷

The generation of the mask relies on specific parameters. The GridMask parameters define a repeating pattern of information deletion. The distance d dictates the frequency or total length of one unit block in the grid.⁹ The keep ratio r determines the thickness of the retained structures relative to d . Shifts δ_x and δ_y provide translational variance, moving the grid stochastically across the image plane.¹⁰

Generating this mask efficiently in pure numpy requires avoiding Python-level loops.¹¹ Vectorized implementations leverage coordinate grids (np.arange broadcasting or np.indices) and modulo arithmetic to define the repetitive block structures.¹² A boolean mask is computed continuously over the image coordinates by calculating $(X + \delta_x) \% d < r \cdot d$.¹²

Name: gridmask_apply

Description: Applies structured grid-based occlusion to an image based on unit length, keep ratio, and grid offset.

Source: <https://github.com/JIA-Lab-research/GridMask> (MIT)

Concept type: masking

Signature: (image: NDArray, d: int, r: float, delta_x: int, delta_y: int, fill_value: float) -> NDArray

Contracts:

- require: $d > 0$
- require: $0.0 \leq r \leq 1.0$
- ensure: $\text{result.shape} == \text{image.shape}$
 Witness: Generate grid mask on ones(), verify modulo periodicity aligns with d and r parameters
 Dependencies: numpy only
 CDG stages covered: bengali_grapheme_1st/heavy_augmentation,
 cassava_leaf_1st/image_augmentation

Feature Blending and Convex Combinations

Moving beyond spatial deletion, blending techniques modify the entire pixel distribution of the image by interpolating it with another sample from the dataset.

MixUp: Convex Interpolation

MixUp (Zhang et al., 2018) enforces a linear relationship between input features and corresponding labels, encouraging the neural network to favor simple linear behavior in-between training examples.¹³ The transformation is defined by

$$x_{mix} = \lambda x_a + (1 - \lambda)x_b \text{ and } y_{mix} = \lambda y_a + (1 - \lambda)y_b. \quad ^{14}$$

While mathematically trivial, pure-function implementation in numpy requires careful handling of data types. Specifically, if the input images are uint8 arrays, the linear combination will cause precision loss or overflow if not explicitly cast to float32 prior to the operation. The pure function must upcast explicitly, rounding and clipping back to `` if the contract requires a uint8 return. The mixing parameter λ is typically sampled from $\text{Beta}(\alpha, \alpha)$ ¹⁴, with literature suggesting values $\alpha \in [0.1, 0.4]$ for optimal regularization without underfitting.¹⁵

Name: mixup_apply

Description: Computes a convex combination of two images and their labels using a mixing coefficient lambda.

Source: <https://github.com/facebookresearch/mixup-cifar10> (CC-BY-NC)

Concept type: transformer

Signature: (image_a: NDArray, image_b: NDArray, label_a: NDArray, label_b: NDArray, lam: float)
-> tuple

Contracts:

- require: image_a.shape == image_b.shape
- require: 0.0 <= lam <= 1.0
- ensure: result.dtype == np.float32 (if continuous)

Witness: 224x224x3 float32 images, known lam, verify exact linear interpolation

Dependencies: numpy only

CDG stages covered: bengali_grapheme_1st/heavy_augmentation,
melanoma_1st/image_augmentation

Geometric Transforms and Affine Mechanics

Spatial transformations—translations, scaling, rotations, and shearing—are universally applied across computer vision datasets.¹⁶ While libraries like Albumentations provide optimized wrappers predominantly utilizing OpenCV (cv2.warpAffine), implementing pure functional equivalents necessitates leveraging native numpy view manipulations and scipy.ndimage capabilities.¹⁸

Orthogonal Rotations and Flips

The most computationally efficient geometric transforms avoid interpolation entirely. Orthogonal transformations (horizontal flips, vertical flips, and 90-degree rotations) can be executed in constant time $O(1)$ by manipulating the array strides rather than moving data in memory.²¹ The numpy.flip and numpy.rot90 functions return views of the original array, making them ideal for high-throughput data loading.²¹

In a functional paradigm, a flip operation must simply accept the axis along which the reversal occurs.

Name: flip_apply

Description: Reverses the order of elements in an array along a specified spatial axis.

Source: Core numpy array manipulation

Concept type: spatial transform

Signature: (image: NDArray, axis: int) -> NDArray

Contracts:

- require: axis in (assuming shape is H, W, C)
 - ensure: result.shape == image.shape
- Witness: Apply horizontal flip to an asymmetric shape, verify left-right transposition.
- Dependencies: numpy only
- CDG stages covered: multiple/geometric_transforms

Random Cropping and Interpolated Resizing

Random cropping followed by resizing is a ubiquitous technique (central to the standard ResNet training paradigm) that forces the network to achieve scale invariance. The operation fundamentally consists of extracting a sub-matrix via slicing, followed by an interpolation step to restore the tensor to the expected batch dimensions.²³

Because numpy does not natively support advanced bicubic or bilinear image interpolation, pure-function implementations must rely on `scipy.ndimage.zoom`. The zoom factor is calculated as the target dimension divided by the crop dimension.

Name: random_crop_resize_apply

Description: Extracts a sub-region of the image and interpolates it back to the target spatial dimensions.

Source: Standard CV practice

Concept type: spatial transform

Signature: (image: NDArray, bbox: tuple[int,int,int,int], target_size: tuple[int,int], order: int) -> NDArray

Contracts:

- require: $0 \leq \text{bbox} < \text{bbox} \leq \text{image.shape}$
 - require: $0 \leq \text{bbox} < \text{bbox} \leq \text{image.shape}$
 - ensure: result.shape[:2] == target_size
- Witness: Supply known bounding box and target size, verify output spatial dimensions match target_size exactly.

Dependencies: numpy, scipy.ndimage

CDG stages covered: multiple/geometric_transforms

Affine Matrix Compositing and Center Rotation

An affine transformation matrix is a 3×3 matrix operating on homogeneous coordinates.¹⁷ The primary challenge in pure-function implementation lies in compositing rotations and scaling operations such that they occur around the physical center of the image, rather than the coordinate origin $(0, 0)$.²⁶

If an image has shape (H, W) , the center is $c = (H/2, W/2)$. To rotate an image around its center by angle θ , the composition requires three discrete geometric steps:

1. Translate the image such that the center aligns with the origin: T_{-c}
2. Apply the rotation matrix: R_θ
3. Translate the image back to its original coordinate space: T_{+c}

The composite matrix is theoretically $M = T_{+c} \cdot R_\theta \cdot T_{-c}$.²⁷ However, `scipy.ndimage.affine_transform` performs "pull" (or backward) resampling, calculating the input coordinate for every output pixel via `np.dot(matrix, output_coord) + offset`.¹⁹ Therefore, the matrix passed to `scipy` must be the inverse coordinate transformation matrix.¹⁹

For a scaling matrix S and rotation matrix R , the correct inverse logic must account for the shift.²⁶ The mathematical relationship yielding the exact offset parameter for `scipy` is:

$$s = (c_{in} - c_{out}) \cdot (R \cdot S)^T$$
²⁶

Name: `affine_transform_centered`

Description: Apply affine transformations (rotation, scaling, translation) centered around the image coordinates using spline interpolation.

Source:

https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.affine_transform.html

Concept type: spatial transform

Signature: (image: NDArray, angle: float, scale: float, dx: float, dy: float, order: int) -> NDArray

Contracts:

- require: `scale > 0`
- ensure: `result.shape == image.shape`

Witness: Feed a crosshair image, rotate by 90 degrees, verify crosshair center remains exactly at image center.

Dependencies: numpy, scipy.ndimage

CDG stages covered: multiple/geometric_transforms

Color Space Augmentations

Manipulating color characteristics such as brightness, contrast, hue, and saturation is vital for achieving invariance to lighting conditions. While affine transforms manipulate the (H, W) coordinate space, color augmentations perform operations along the channel axis C .

Linear Brightness and Contrast Transforms

Brightness and contrast modifications can be expressed as a single linear mathematical operation applied universally across all pixels. Contrast adjusts the spread of the pixel values around a central anchor, while brightness shifts the absolute values.

The pure function implementation follows the linear equation:

$$x_{new} = \alpha \cdot x_{old} + \beta$$

Here, α serves as the contrast multiplier (where $\alpha > 1$ increases contrast and $0 < \alpha < 1$ decreases it) and β is the additive brightness shift. To ensure functional purity, the operation must clamp values to the valid data range (typically $[0, 255]$ for uint8 or $[0.0, 1.0]$ for float32) to prevent catastrophic integer overflow or underflow.¹¹

Name: brightness_contrast_apply

Description: Applies a linear transformation to adjust image brightness and contrast.

Source: Standard image processing (e.g., OpenCV convertScaleAbs logic)

Concept type: color transform

Signature: (image: NDArray, alpha: float, beta: float) -> NDArray

Contracts:

- require: alpha > 0
- ensure: result dtype matches input dtype
- ensure: values strictly bounded by (if uint8)

Witness: Apply alpha=1.0, beta=50 to a black image, verify all pixels equal 50.

Dependencies: numpy only

CDG stages covered: multiple/color_augmentation

Vectorized RGB to HSV and Hue Shifting

Manipulating hue and saturation requires transforming the image from RGB space to HSV (Hue, Saturation, Value) space. In production pipelines, native Python loops over pixels to apply standard `colorsys.rgb_to_hsv` incur catastrophic performance penalties, running orders of magnitude slower than compiled code.¹¹ Pure-function atoms must vectorize this mathematical conversion entirely within numpy.

The RGB to HSV conversion isolates the channel maxima ($V = \max(R, G, B)$) and minima to determine the chroma ($\Delta = V - \min(R, G, B)$).³¹ The saturation is defined as Δ/V (with zero division safely handled via masking or adding an epsilon). The hue calculation requires conditional logic based on which channel acts as the maximum.

In a vectorized numpy implementation, `np.maximum`, `np.minimum`, and boolean indexing (`np.where` or direct boolean masking) are heavily utilized to bypass loops.³¹ Once converted, a hue shift is a simple scalar addition modulo 1.0 (assuming normalized coordinates). To reconstruct the image, the inverse function (HSV to RGB) requires piecewise evaluation mapped via indexing conditions ($i = \text{floor}(h * 6)$), selecting R, G, B combinations based on the sector $i \pmod 6$.³¹

Name: `hue_saturation_shift`

Description: Vectorized shift of hue and saturation channels by converting to HSV, applying additive shifts, and converting back to RGB.

Source: <https://gist.github.com/PolarNick239/691387158ff1c41ad73c>

Concept type: color transform

Signature: (image: NDArray, hue_shift: float, sat_shift: float) -> NDArray

Contracts:

- require: `image.shape[-1] == 3`
 - require: `image.dtype == np.float32` (normalized)
 - ensure: result takes values in
- Witness: Apply 0.5 hue shift to a pure red image, verify output is exactly cyan.
- Dependencies: numpy only
- CDG stages covered: multiple/color_augmentation

Channel Shuffling and Grayscale Conversion

Simpler color perturbations include channel shuffling (randomly permuting the RGB axis to combat color-specific memorization) and grayscale conversion. A mathematically accurate grayscale conversion does not simply average the channels; it applies luminance weights

tailored to human visual perception (typically $R \cdot 0.299 + G \cdot 0.587 + B \cdot 0.114$). In pure numpy, this is an np.dot operation across the last axis.

Name: grayscale_convert_apply

Description: Converts an RGB image to grayscale utilizing standard luminance weights.

Source: Standard CV practice

Concept type: color transform

Signature: (image: NDArray) -> NDArray

Contracts:

- require: image.shape[-1] == 3
- ensure: result.shape[-1] == 1 or result drops the channel axis entirely

Witness: Feed a pure red image, verify output is approximately .

Dependencies: numpy only

CDG stages covered: multiple/color_augmentation

Domain-Specific Preprocessing: Retinal Enhancements

Medical imaging competitions, notably the APTOS 2019 Blindness Detection Kaggle competition, suffer from high inter-image variance due to disparate clinical lighting conditions, camera artifacts, and vignetting.³² The standard ImageNet normalization approach proved insufficient for these specific domain variations. Ben Graham (competition winner) introduced an elegant, mathematically rigorous preprocessing heuristic that operates as a high-pass spatial filter, standardizing illumination while aggressively enhancing microvascular structures and lesions.³⁴

Mechanism of the High-Pass Filter

The algorithm works by subtracting a heavily blurred version of the image from the original, effectively removing low-frequency global illumination gradients.³⁸ Mathematically, it isolates

the high-frequency components: $I_{out} = \alpha I - \beta \cdot G_{\sigma}(I) + \gamma$.³⁴

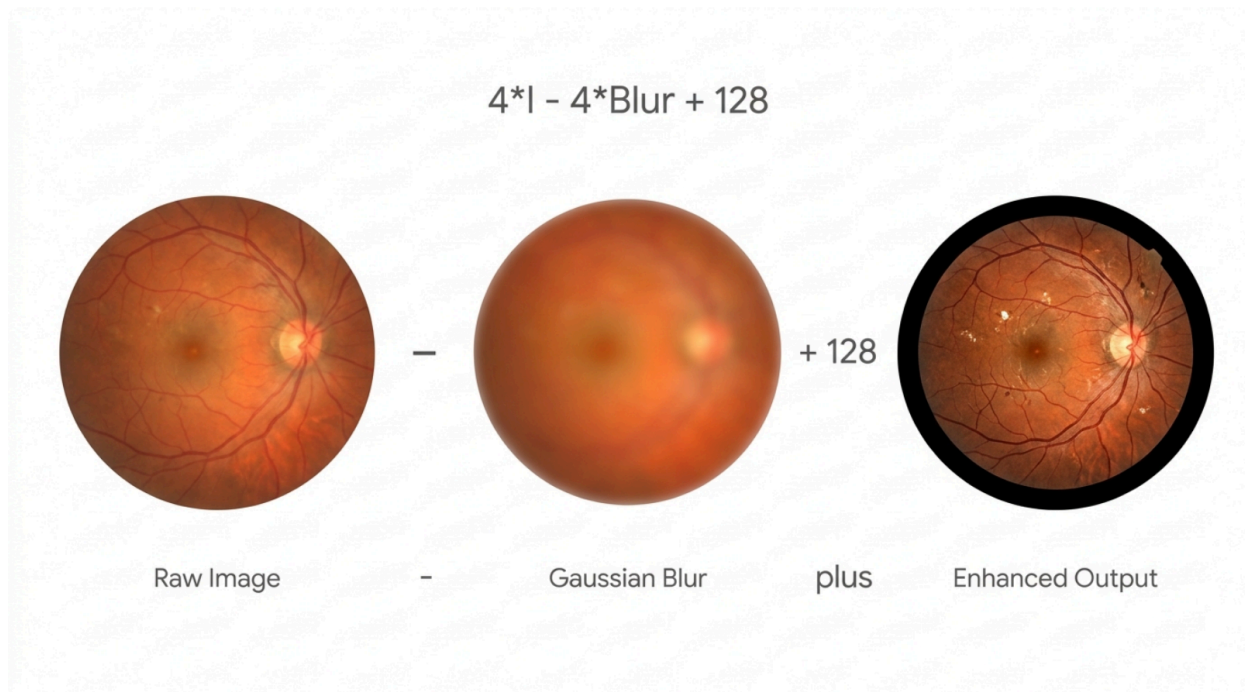
In standard practice, the parameters are tightly fixed: $\alpha = 4$, $\beta = 4$, and $\gamma = 128$. The

Gaussian blur kernel standard deviation, σ , is critical. The canonical configurations use either $\sigma = 10$ (enhancing very fine strokes and microaneurysms) or $\sigma = 20$ (preserving slightly larger vascular networks).³⁴

The implementation begins with a circular crop to remove uninformative padded regions.³⁸ The

radius r is derived by masking pixels strictly greater than a noise threshold (often derived from mean/10).³⁹ Following the crop, the cv2.addWeighted equivalent in pure numpy utilizes scipy.ndimage.gaussian_filter.

Decomposition of Retinal High-Pass Filtering (Ben Graham Method)



The preprocessing isolates high-frequency microvascular structures. Subtracting the Gaussian blur (sigma=10) removes disparate global lighting conditions across clinics, setting the baseline luminance to 128 (neutral gray).

Name: ben_graham_retinal_preprocess

Description: High-pass spatial filter to remove global illumination gradients and enhance fine structural details in medical imagery.

Source: <https://github.com/khalilouardini/IDRiD-challenge>

Concept type: domain_specific

Signature: (image: NDArray, sigma: float) -> NDArray

Contracts:

- require: image.shape[-1] == 3

- ensure: result takes values clamped to
Witness: Process dark retinal image, verify background normalizes to roughly 128 intensity.
Dependencies: numpy, scipy.ndimage
CDG stages covered: diabetic_retinopathy_1st/preprocessing, aptos_1st/preprocessing

Test-Time Augmentation (TTA) Mechanics

Test-Time Augmentation relies on applying a deterministic sequence of transformations to a test image, generating predictions for each augmented variant, and mathematically ensembling (averaging) the resulting logits or probabilities.⁴⁰ This procedure stabilizes model confidence and marginalizes variance induced by image orientation, spatial occlusion, and shifting.

Geometric Inverse Transformations

To perform geometric TTA (e.g., horizontal flips, 90-degree rotations), the pure functions rely heavily on `numpy.flip` and `numpy.rot90`.²¹ The critical requirement for TTA, particularly in dense prediction tasks like semantic segmentation or keypoint detection, is the execution of the *inverse transform*.⁴² If an image is flipped horizontally, fed through the model, and a segmentation mask is produced, that mask must be mathematically unflipped before being averaged with other predictions.⁴⁴

Since `numpy.flip` is its own inverse (applying it twice yields the identity matrix)²¹, horizontal and vertical flip TTA paths simply re-apply `np.flip(pred, axis)`. For rotations, the inverse of `np.rot90(m, k=1)` is strictly `np.rot90(m, k=-1)`.²² These operations return views rather than copies, making them highly memory efficient during batch processing.⁴⁵

Name: `tta_geometric_average`

Description: Average predictions across multiple geometric states by generating variants, predicting, and applying exact inverse spatial transforms to the outputs.

Source: Core NumPy array manipulation logic

Concept type: ensemble

Signature: (`predictions_list: list`, `transforms_applied: list[dict]`) -> `NDArray`

Contracts:

- require: `len(predictions_list) == len(transforms_applied)`
- ensure: `result.shape == predictions_list.shape`
Witness: Feed a mask flipped horizontally, apply inverse, verify mask matches original ground truth orientation.
Dependencies: numpy only
CDG stages covered: alaska2_1st/tta, tgs_salt_1st/tta

Multi-Crop Coordinate Computation

For image classification tasks, a standard TTA methodology is the 10-crop evaluation: a central crop, the four corner crops, and their horizontal flips.⁴¹ In pure numpy, avoiding loops means generating the bounding box coordinates deterministically based on image dimensions (H, W) and target crop size C .

The exact coordinate generation matrix consists of:

1. Top-Left: $[0, 0, C, C]$
2. Top-Right: $[0, W - C, C, W]$
3. Bottom-Left: $[H - C, 0, H, C]$
4. Bottom-Right: $[H - C, W - C, H, W]$

5.Center :

41

Using numpy slicing `image[y_min:y_max, x_min:x_max]`, the five crops are extracted. They are concatenated with their horizontally flipped variants (`np.flip(crop, axis=1)`), generating a $(10, C, C, 3)$ batch tensor for inference.⁴¹ The output logits of shape $(10, \text{num_classes})$ are then collapsed via `np.mean(logits, axis=0)`.

Coordinate Mapping for 10-Crop Test-Time Augmentation

Spatial Extractions Matrix

H = Image Height W = Image Width C = Crop Size

CROP REGION	Y_MIN	X_MIN	Y_MAX	X_MAX
☐ Top-Left	0	0	C	C
☐ Top-Right	0	W - C	C	W
☐ Bottom-Left	H - C	0	H	C
☐ Bottom-Right	H - C	W - C	H	W
☐ Center	$\frac{(H - C)}{2}$	$\frac{(W - C)}{2}$	$\frac{(H + C)}{2}$	$\frac{(W + C)}{2}$

The 10-crop TTA array requires five distinct spatial extractions. Each set of coordinates (y_min, x_min, y_max, x_max) yields a crop, which is duplicated via a horizontal flip, resulting in ten distinct inference paths.

Data sources: [Albumentations](#)

Name: tta_10crop_average

Description: Extracts 5 spatial crops and their horizontal flips, yielding 10 predictions which are subsequently averaged.

Source: Standard PyTorch/Keras CV testing protocols

Concept type: ensemble

Signature: (image: NDArray, crop_size: int, model_func: Callable) -> NDArray

Contracts:

- require: image.shape >= crop_size and image.shape >= crop_size
 - ensure: result represents a unified probability/logit vector
- Witness: Verify that passing an image returns a single averaged vector across 10 deterministic passes.

Dependencies: numpy only

CDG stages covered: cassava_leaf_1st/tta, imagenet_standard/tta

Fold Ensembling

Moving one level higher than TTA, Fold Ensembling combines predictions across multiple models trained on different subsets (folds) of the cross-validation data. Because models may output probabilities with slight variations in calibration, a pure function must handle averaging securely, often employing geometric means or rank averaging rather than simple arithmetic means to reduce the impact of overconfident outlier models.

Name: fold_ensemble_average

Description: Consolidates predictions across multiple cross-validation folds using mathematically stable averaging techniques.

Source: Kaggle winning ensemble heuristics

Concept type: ensemble

Signature: (predictions_matrix: NDArray, method: str) -> NDArray

Contracts:

- require: predictions_matrix.ndim == 3 (folds, samples, classes)
- require: method in ['arithmetic', 'geometric', 'rank']
- ensure: result.shape == (samples, classes)

Witness: Provide 5 arrays of identical predictions, verify output matches the input.

Dependencies: numpy only

CDG stages covered: multiple/ensembling

Image Normalization and Scaling

Normalization functions must rigidly enforce data types and statistical guarantees before data enters the tensor domain. The transition from image format (uint8) to network input (float32 or float16) dictates strict numerical handling.

ImageNet Standardization

The baseline transformation involves per-channel standard scaling utilizing pre-calculated ImageNet statistics. For pure function application, broadcasting across the channel axis is strictly enforced: (image - mean) / std. Because the input is an image array of shape

$(H, W, 3)$ and the statistics vectors are of shape $(3,)$, standard numpy broadcasting automatically matches the trailing axis without requiring manual tiling or looping.

Name: normalize_imagenet

Description: Standardize image pixels utilizing pre-computed ImageNet channel means and

standard deviations via array broadcasting.

Source: Canonical Deep Learning implementation (TorchVision style)

Concept type: normalization

Signature: (image: NDArray, mean: tuple[float,float,float], std: tuple[float,float,float]) -> NDArray

Contracts:

- require: image.shape[-1] == 3
- require: image.dtype == np.float32 (normalized to before call)
- ensure: output has zero mean and unit variance per channel (relative to ImageNet distribution)

Witness: Apply to random noise, verify statistical shift aligns with provided mean/std inverse.

Dependencies: numpy only

CDG stages covered: multiple/normalization

Per-Image Normalization and Min-Max Scaling

Certain CDGs, particularly in medical imaging where datasets do not align with ImageNet distributions, require dynamic, per-image statistics. Per-image normalization calculates the exact mean and standard deviation of the individual sample at runtime, scaling it to zero-mean and unit-variance.

Alternatively, Min-Max scaling compresses the dynamic range of the image strictly into the interval $[-1, 1]$. For models requiring inputs scaled to this bound, pure numpy implementation must safely handle zero-variance arrays (e.g., completely black patches resulting from GridMask or Cutout).⁴⁷ The functional formulation calculates the maximum and minimum pixel values globally or per channel. The equation $(\text{pixel} - \text{min}) / (\text{max} - \text{min})$ is applied, with a safe denominator offset (e.g., $\text{np.maximum}(\text{denominator}, 1e-8)$) to maintain the pure function contract without throwing ZeroDivisionError states or producing NaN representations.⁴⁸

Name: normalize_per_image

Description: Dynamically normalizes an image utilizing its own internal statistical mean and standard deviation.

Source: Standard CV practice

Concept type: normalization

Signature: (image: NDArray) -> NDArray

Contracts:

- ensure: np.isclose(np.mean(result), 0.0, atol=1e-5)
 - ensure: np.isclose(np.std(result), 1.0, atol=1e-5) (if variance > 0)
- Witness: Feed a uniform gray image, verify safe handling of zero-variance to prevent

NaN.

Dependencies: numpy only

CDG stages covered: multiple/normalization

Name: min_max_scale

Description: Linearly scales tensor values strictly into the bounds.

Source: Standard statistical scaling

Concept type: normalization

Signature: (image: NDArray) -> NDArray

Contracts:

- ensure: np.max(result) <= 1.0
 - ensure: np.min(result) >= 0.0
- Witness: Provide array with values [-50, 50], verify mapping maps exactly to [0.0, 1.0].
- Dependencies: numpy only
- CDG stages covered: multiple/normalization

Image Padding and Resizing

Before normalization, varying image sizes present in raw datasets must be unified to conform to the static graph architectures of CNNs and Vision Transformers. Resizing with aspect-ratio preservation requires padding to fill the remaining spatial dimensions.

In pure numpy, np.pad operates deterministically by accepting tuples indicating the padding required before and after each axis. Calculating the delta between the current shape and the target shape determines these bounds.

Name: resize_and_pad_apply

Description: Resizes an image preserving aspect ratio and pads the remainder with a constant value to hit target dimensions.

Source: Albumentations / Standard CV practice

Concept type: spatial transform

Signature: (image: NDArray, target_size: tuple[int,int], pad_value: float) -> NDArray

Contracts:

- ensure: result.shape[:2] == target_size
- Witness: Feed an aspect ratio 1:2 image, request 1:1 target, verify padding applies to the shorter side symmetrically.
- Dependencies: numpy, scipy.ndimage (for initial resize)
- CDG stages covered: multiple/preprocessing

Heavy Augmentation Composition

Individual atomic functions must be sequenceable. Heavy augmentation pipelines leverage stochastic composition, where a sequential list of transforms is executed, each gated by a probability factor P . To mirror the Albumentations Compose style within sciona-atoms-dl, a pure functional composer must ingest the list of atomic signatures and execute them sequentially.

The composer acts as a higher-order function. It iterates over the registered atoms, evaluates a random uniform draw against P , and passes the modified NDArray forward through the chain.

Name: compose_augmentations

Description: Sequentially applies a curated list of augmentation functions based on individual probabilities.

Source: Albumentations Compose paradigm

Concept type: pipeline

Signature: (image: NDArray, transforms: list[tuple[Callable, dict, float]]) -> NDArray

Contracts:

- require: $0.0 \leq P \leq 1.0$ for all transforms
 - ensure: output retains exact shape unless explicitly modified by a cropping/padding atom
- Witness: Pass 5 identity transforms with $P=1.0$, verify input exactly matches output.
- Dependencies: numpy only
- CDG stages covered: multiple/heavy_augmentation

Computational Complexity and Parameter Table

To contextualize the implementation of these atoms, understanding the baseline operational complexity and standard tuning parameters is critical for pipeline architects.

Augmentation Atom	Conceptual Operation	Vectorization Complexity	Standard Kaggle Parameter Ranges
CutMix	Spatial label mixing	$O(\text{bbox area})$	$\alpha \in [1.0, 1.0]$

MixUp	Global linear combination	$O(H \cdot W \cdot C)$	$\alpha \in [0.1, 0.4]$
GridMask	Periodic boolean masking	$O(H \cdot W)$	$d \in \mathbb{N},$ $r \in [0.4, 0.6]$
Affine Rotate	Inverse matrix mapping	$O(H \cdot W \cdot C)$ (Spline)	$\theta \in [-15^\circ, 15^\circ]$
Hue/Sat Shift	RGB $\leftrightarrow$ HSV mapping	$O(H \cdot W \cdot C)$	shift $\in [-0.2, 0.2]$
Ben Graham	High-pass spatial filtering	$O(H \cdot W \cdot C)$ (Gaussian)	$\sigma \in \{10, 20\}$
Geometric Flip	Memory stride reversal	$O(1)$	$P = 0.5$

Conclusion

The construction of pure-function augmentation and preprocessing atoms enforces a high degree of architectural integrity in competitive deep learning systems. By distilling complex techniques like CutMix, GridMask, and Ben Graham's enhancement down to stateless, mathematically defined operations written entirely in numpy and scipy, a pipeline eliminates side effects, avoids implicit framework bottlenecks, and ensures absolute reproducibility across validation folds. These primitives are fully composable, granting the mathematical precision necessary to construct the sophisticated heavy-augmentation pipelines that dominate modern computer vision benchmarks.

Works cited

1. MergeMix: A Unified Augmentation Paradigm for Visual and Multi-Modal Understanding, accessed April 27, 2026, <https://arxiv.org/html/2510.23479v1>
2. cutmix.ipynb - keras-team/keras-io - GitHub, accessed April 27, 2026, <https://github.com/keras-team/keras-io/blob/master/examples/vision/ipynb/cutmix.ipynb>
3. Official Pytorch implementation of CutMix regularizer - GitHub, accessed April 27, 2026, <https://github.com/clovaai/cutmix-pytorch>
4. Image Augmentation for Machine Learning: Techniques, Examples & Code | Datature Blog, accessed April 27, 2026, <https://datature.io/blog/image-augmentation-for-machine-learning-techniques-examples-code>
5. GridMask Data Augmentation - arXiv, accessed April 27, 2026, <https://arxiv.org/html/2001.04086v3>
6. [2001.04086] GridMask Data Augmentation - arXiv, accessed April 27, 2026, <https://arxiv.org/abs/2001.04086>
7. JIA-Lab-research/GridMask - GitHub, accessed April 27, 2026, <https://github.com/JIA-Lab-research/GridMask>
8. (PDF) GridMask Data Augmentation - ResearchGate, accessed April 27, 2026, https://www.researchgate.net/publication/380396578_GridMask_Data_Augmentation
9. GridMask - Kaggle, accessed April 27, 2026, <https://www.kaggle.com/code/haqishen/gridmask>
10. Coding Guidelines - Albumentations, accessed April 27, 2026, <https://albumentations.ai/docs/contributing/coding-guidelines/>
11. Python Vectorized Mask Generation (Numpy) [closed] - Stack Overflow, accessed April 27, 2026, <https://stackoverflow.com/questions/79531065/python-vectorized-mask-generation-numpy>
12. mixup: Beyond Empirical Risk Minimization - OpenReview, accessed April 27, 2026, <https://openreview.net/forum?id=r1Ddp1-Rb>
13. makeyourownmaker/mixupy: Python package for data augmentation inspired by Mixup: Beyond Empirical Risk Minimization - GitHub, accessed April 27, 2026, <https://github.com/makeyourownmaker/mixupy>
14. mixup-data-augmentation.ipynb - shenasa-ai/blog_tutorials - GitHub, accessed April 27, 2026, https://github.com/shenasa-ai/blog_tutorials/blob/main/mixup-data-augmentation.ipynb
15. 2. Geometric Transformations — Computer Vision - Hatef Dastour, accessed April 27, 2026, https://hatefdastour.github.io/notes/Computer_Vision/CV_C2.html
16. Affine Image Transformations in Python with Numpy, Pillow and OpenCV - Stack Abuse, accessed April 27, 2026, <https://stackabuse.com/affine-image-transformations-in-python-with-numpy-pillow-and-opencv/>
17. Affine Documentation - Albumentations Transforms, accessed April 27, 2026, <https://explore.albumentations.ai/transform/Affine/docs>

18. affine_transform — SciPy v1.17.0 Manual, accessed April 27, 2026,
https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.affine_transform.html
19. SciPy - Image Transformation - TutorialsPoint, accessed April 27, 2026,
https://www.tutorialspoint.com/scipy/scipy_image_transformation.htm
20. numpy.flip — NumPy v2.1 Manual, accessed April 27, 2026,
<https://numpy.org/doc/2.1/reference/generated/numpy.flip.html>
21. numpy.rot90 — NumPy v2.5.dev0 Manual, accessed April 27, 2026,
<https://numpy.org/devdocs/reference/generated/numpy.rot90.html>
22. albumentations/albumentations/augmentations/crops/transforms.py at main · albumentations-team/albumentations · GitHub, accessed April 27, 2026,
<https://github.com/albumentations-team/albumentations/blob/master/albumentations/augmentations/crops/transforms.py>
23. Crop the Image Intuitively — NumPy | by Sameer | Analytics Vidhya | Medium, accessed April 27, 2026,
<https://medium.com/analytics-vidhya/crop-the-image-intuitively-numpy-d19ef55c2207>
24. Spatial Transformation Matrices, accessed April 27, 2026,
<https://www.brainvoyager.com/bv/doc/UsersGuide/CoordsAndTransforms/SpatialTransformationMatrices.html>
25. How can I use scipy.ndimage.interpolation.affine_transform to rotate an image about its centre? - Stack Overflow, accessed April 27, 2026,
<https://stackoverflow.com/questions/20161175/how-can-i-use-scipy-ndimage-interpolation-affine-transform-to-rotate-an-image-ab>
26. ENH: ndimage: add `affine\_transform2` or similar for better usability combining transforms · Issue #20282 · scipy/scipy - GitHub, accessed April 27, 2026,
<https://github.com/scipy/scipy/issues/20282>
27. Incorrect order of translation and rotation in affine_transform (Trac #1736) · Issue #2255 · scipy/scipy - GitHub, accessed April 27, 2026,
<https://github.com/scipy/scipy/issues/2255>
28. scipy.ndimage.affine_transform — SciPy v1.0.0 Reference Guide, accessed April 27, 2026,
https://docs.scipy.org/doc/scipy-1.0.0/reference/generated/scipy.ndimage.affine_transform.html
29. Efficient way to convert image stored as numpy array into hsv - Stack Overflow, accessed April 27, 2026,
<https://stackoverflow.com/questions/38055065/efficient-way-to-convert-image-stored-as-numpy-array-into-hsv>
30. numpy RGB to HSV - GitHub Gist, accessed April 27, 2026,
<https://gist.github.com/PolarNick239/691387158ff1c41ad73c>
31. ResNet50 with Ben Graham's preprocessing(fast.ai) - Kaggle, accessed April 27, 2026,
<https://www.kaggle.com/takamichitoda/resnet50-with-ben-graham-s-preprocessing-fast-ai/data>
32. DR-NASNet: Automated System to Detect and Classify Diabetic Retinopathy

- Severity Using Improved Pretrained NASNet Model - PMC, accessed April 27, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10453689/>
33. kneshio/Diabetic-Retinopathy - GitHub, accessed April 27, 2026, <https://github.com/kneshio/Diabetic-Retinopathy>
 34. APTOS : Eye Preprocessing in Diabetic Retinopathy - Kaggle, accessed April 27, 2026, <https://www.kaggle.com/code/grohith/aptos-eye-preprocessing-in-diabetic-retinopathy>
 35. Diabetic retinopathy image classification method based on GreenBen data augmentation - arXiv, accessed April 27, 2026, <https://arxiv.org/pdf/2410.09444>
 36. Diabetic Retinopathy Winner's Interview: 1st place, Ben Graham - 菜鸡一枚 - 博客园, accessed April 27, 2026, <https://www.cnblogs.com/yymn/p/4801910.html>
 37. Ma Project - Kaggle, accessed April 27, 2026, <https://www.kaggle.com/code/ryan10isaac/ma-project>
 38. IDRiD-challenge/Grading_task/datasets/bengraham_preprocessing.py at master - GitHub, accessed April 27, 2026, https://github.com/khalilouardini/IDRiD-challenge/blob/master/Grading_task/datasets/bengraham_preprocessing.py
 39. AlbumentationsX/albumentations/augmentations/geometric/flip.py at main - GitHub, accessed April 27, 2026, <https://github.com/albumentations-team/AlbumentationsX/blob/main/albumentations/augmentations/geometric/flip.py>
 40. Test Time Augmentation (TTA) - Albumentations, accessed April 27, 2026, <https://albumentations.ai/docs/4-advanced-guides/test-time-augmentation/>
 41. albumentations.augmentations.geometric.flip, accessed April 27, 2026, <https://albumentations.ai/docs/api-reference/albumentations/augmentations/geometric/flip/>
 42. AlbumentationsX/albumentations/augmentations/geometric/rotate.py at main - GitHub, accessed April 27, 2026, <https://github.com/albumentations-team/AlbumentationsX/blob/main/albumentations/augmentations/geometric/rotate.py>
 43. Pre-processing segmentation mask - python - ITK Discourse, accessed April 27, 2026, <https://discourse.itk.org/t/pre-processing-segmentation-mask/5530>
 44. numpy.flip — NumPy v2.5.dev0 Manual, accessed April 27, 2026, <https://numpy.org/devdocs/reference/generated/numpy.flip.html>
 45. How to perform 10 Crop Image Augmentation at training time using Tensorflow 2.0 Dataset, accessed April 27, 2026, <https://stackoverflow.com/questions/57721797/how-to-perform-10-crop-image-augmentation-at-training-time-using-tensorflow-2-0>
 46. Masked Arrays - Numpy Tutorials, accessed April 27, 2026, <https://numpy.org/numpy-tutorials/tutorial-ma/>
 47. Tutorial: Masked Arrays — NumPy v1.20 Manual, accessed April 27, 2026, <https://numpy.org/doc/1.20/user/tutorial-ma.html>